

Operating Systems

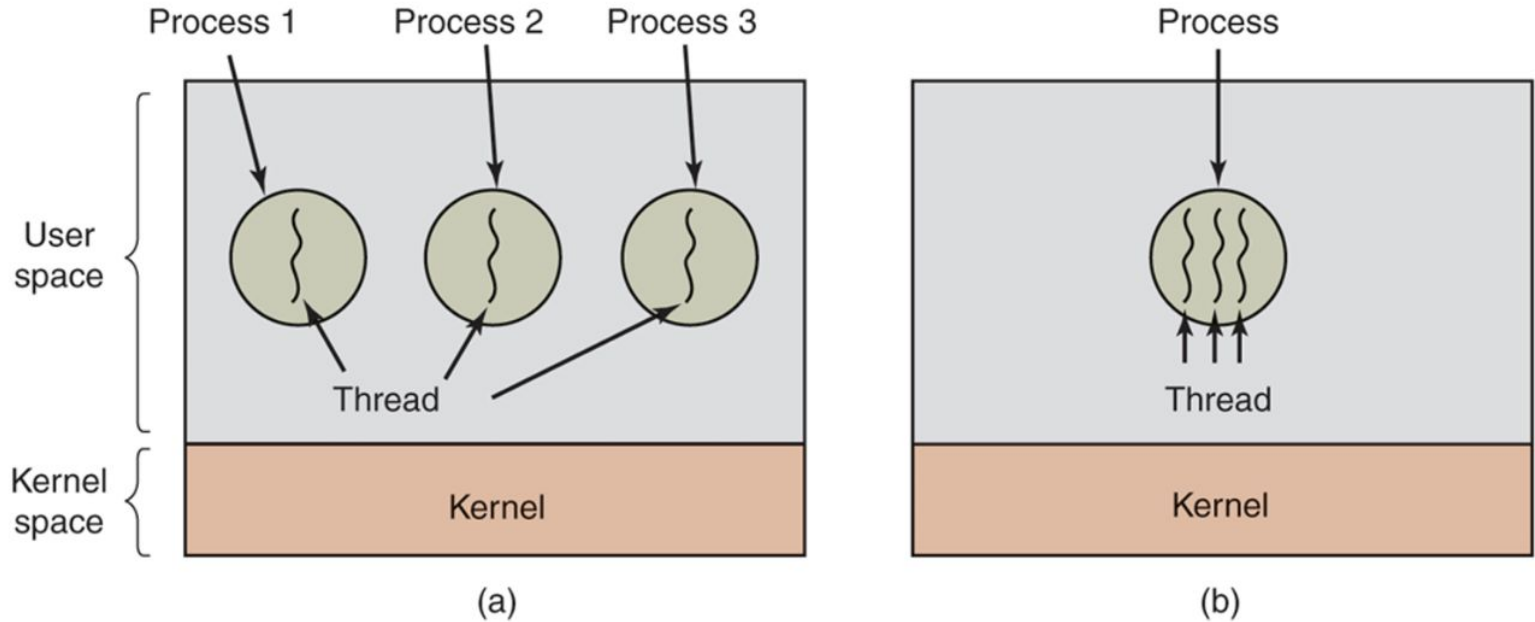
Rowan University

Overview

1. Process & Thread Review
2. Interprocess Communication & Synchronization
3. Race Conditions
4. Semaphores

Recall: Process & Thread Model

Figure 2-10



(a) Three processes each with one thread. (b) One process with three threads.

Example: Shell Forks Process

Figure 10-8

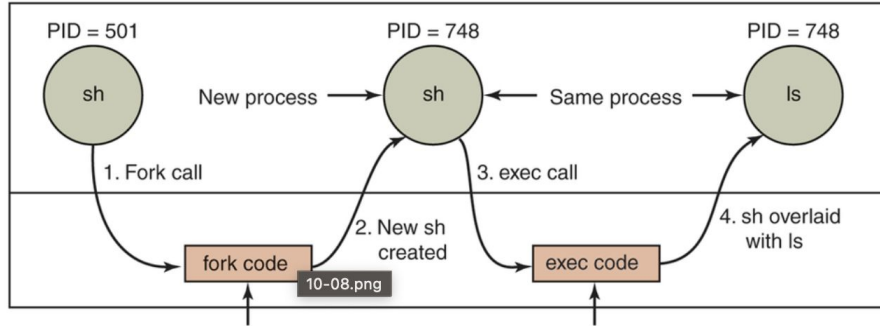


Figure 1-19

```
#define TRUE 1

while (TRUE) {
    type_prompt( );
    read_command(command, parameters);

    if (fork() != 0) {
        /* Parent code. */
        waitpid(-1, &status, 0);
    } else {
        /* Child code. */
        execve(command, parameters, 0);
    }
}
```

/* repeat forever */
/* display prompt on the screen */
/* read input from terminal */

/* fork off child process */

/* wait for child to exit */

/* execute command */

A stripped-down shell. Throughout this book, *TRUE* is assumed to be defined as 1.

Interprocess Communication (IPC)

Issues to consider:

1. Processes need to **communicate** with each other
2. Processes need to **avoid clashing** into each other
3. Processes need to **handle dependencies** on each other

Communication and clashing stem from the dependency on shared memory

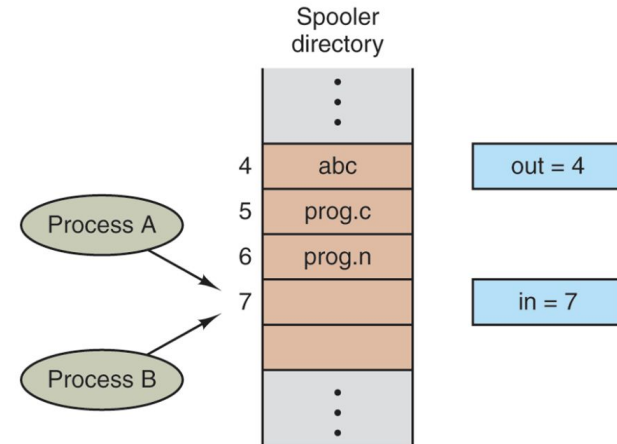
Inter Process Communications (IPC)

- How does one process pass information to another?
- Processes often require synchronization and communication
- Shell pipeline transfers output between processes
 - `history | grep export`
- Challenge: Processes can conflict with each other while sharing resources
- Solution: OS makes certain features available so application programmers can handle sharing resources safe and effectively

Shared Memory Communication

- Consider
 - Two process try to print a file
 - Processes queue files for printing by interacting with the *spooler directory*
 - Files put into the spooler directory are then printed by a *printer daemon*
 - There is a variable `next_free_slot` which processes read to determine where to write to in the spooler directory
- Processes often need to share memory
 - Need to avoid conditions where memory is read/written out of expected sequence

Figure 2-21



Two processes want to access shared memory at the same time.

Race Conditions

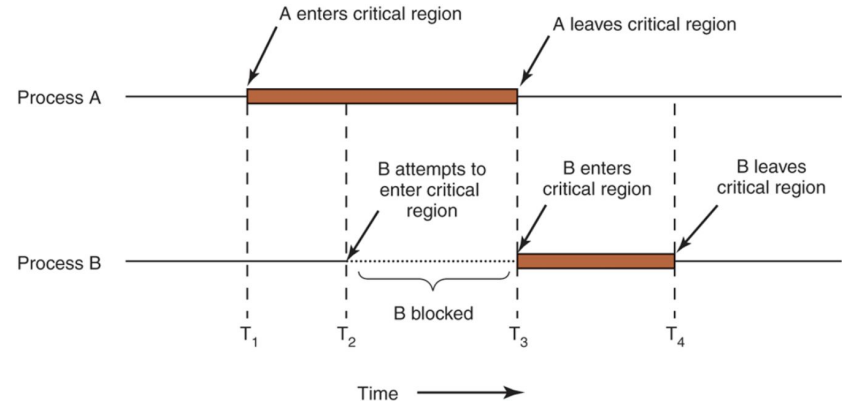
Thread 1	Thread 2		Integer value
			0
read value		←	0
increase value			0
write back		→	1
	read value	←	1
	increase value		1
	write back	→	2

Thread 1	Thread 2		Integer value
			0
read value		←	0
	read value	←	0
increase value			0
	increase value		0
write back		→	1
	write back	→	1

Critical Region

- Implement mutual exclusion to prohibit simultaneous read and write operations
 - If one process accesses shared data others are excluded from access
- Part of the program where shared memory is accessed is called the *critical region*
 - No two processes are simultaneously inside their critical regions

Figure 2-22



Mutual exclusion using critical regions.

Mutual Exclusion: Lock Variables

- Disable Interrupts
 - Single processor system a process disables interrupts before entering its crit. region
- Lock Variables
 - Create a variable that is set before a process proceeds into its CR, others check the lock
- Strict Alternation
 - Keep one process waiting while the other is busy in the critical region, “spin lock”
 - Example below:

Figure 2-23

```
while (TRUE) {  
    while (turn != 0) { }    /* loop */  
    critical_region();  
    turn = 1;  
    noncritical_region();  
}
```

(a)

```
while (TRUE) {  
    while (turn != 1) { }    /* loop */  
    critical_region();  
    turn = 0;  
    noncritical_region();  
}
```

(b)

A proposed solution to the critical-region problem. (a) Process 0. (b) Process 1. In both cases, be sure to note the semicolons terminating the while statements.

Mutual Exclusion: Peterson's Algorithm

Figure 2-24

```
#define FALSE 0
#define TRUE 1
#define N      2                /* number of processes */

int turn;                       /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process);  /* process is 0 or 1 */
{
    int other;                   /* number of the other process */

    other = 1 - process;         /* the opposite of process */
    interested[process] = TRUE;  /* show that you are interested */
    turn = process;              /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)   /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

Mutex

Figure 2-31

Thread call	Description
pthread_mutex_init	Create a mutex
pthread_mutex_destroy	Destroy an existing mutex
pthread_mutex_lock	Acquire a lock or block
pthread_mutex_trylock	Acquire a lock or fail
pthread_mutex_unlock	Release a lock

Some of the Pthreads' calls relating to mutexes.

Mutual Exclusion: Test and Set Lock

Figure 2-25

enter_region:	
TSL REGISTER, LOCK	I copy lock to register and set lock to 1
CMP REGISTER, #0	I was lock zero?
JNE enter_region	I if it was not zero, lock was set, so loop
RET	I return to caller; critical region entered
leave_region:	
MOVE LOCK, #0	I store a 0 in lock
RET	I return to caller

Entering and leaving a critical region using the TSL instruction.

Figure 2-26

enter_region:	
MOVE REGISTER, #1	I put a 1 in the register
XCHG REGISTER, LOCK	I swap contents of register and lock variable
CMP REGISTER, #0	I was lock zero?
JNE enter_region	I if it was non zero, lock was set, so loop
RET	I return to caller; critical region entered
leave_region:	
MOVE LOCK, #0	I store a 0 in lock
RET	I return to caller

Entering and leaving a critical region using the XCHG instruction.

Producer Consumer Model

- Common for Distributed Systems (Apache Kafka)
- Two processes share a common, fixed-size buffer
 - Can be scaled to many processes
 - Producer will sleep if the buffer is full
- Part of Sleep & Wakeup approach to IPC

Figure 2-27

```
#define N 100
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

/ number of slots in the buffer */*
/ number of items in the buffer */*

/ repeat forever */*
/ generate next item */*
/ if buffer is full, go to sleep */*
/ put item in buffer */*
/ increment count of items in buffer */*
/ was buffer empty? */*

/ repeat forever */*
/ if buffer is empty, got to sleep */*
/ take item from buffer */*
/ decrement count of items in buffer */*
/ was buffer full? */*
/ print item */*

The producer-consumer problem with a fatal race condition.

Semaphores

- Similar to a binary mutex lock except it acts as a queue of waiting processes
- Semaphore is an integer
 - Equal to the number of process waiting to be woken up
- Process sleeps when semaphore is 0 and `down` is called on it
 - `down(&full)`

Figure 2-28

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

```
/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */
```

```
/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */
```

```
/* infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */
```

Xinu Galileo

- Examine the Process implementation in Xinu
- Link to source code in Github has been added to Canvas
- <https://github.com/real-xinu/xinu-galileo/blob/master/include/process.h>
- <https://github.com/real-xinu/xinu-galileo/blob/master/include/kernel.h>

Questions

- On all current computers, at least part of the interrupt handlers are written in assembly language. Why?
- Explain how a Web browser can utilize the concept of threads to improve performance.